Technische Dokumentation – Anmeldeformular mit Next.js, TypeScript, Zod und Strapi

Verwendete Technologien



TypeScript

Programmiersprache mit Typprüfung



Zod

Validierung und Überprüfung der Formulardaten

Strapi

Datenbankverwaltung und API



Axios

Kommunikation zwischen Next.js und Strapi



TanStack React Query

Verwaltung der Datenabfragen und Änderungen



React Hook Form

Verwaltung der Formulare

1. Aufbau und Funktionsweise der Anwendung

Deine Anwendung besteht aus mehreren Komponenten, die miteinander kommunizieren.

Grundsätzlich funktioniert das Ganze so:

Next.js / React

Benutzeroberfläche und Formulare

React Hook Form + Zod

Formularverwaltung und Datenvalidierung

TanStack React Query + Axios

Datenabfragen und API-Kommunikation

Strapi API

Empfängt und verarbeitet Anfragen

Datenbank

Speichert die Anmeldedaten dauerhaft

Die einzelnen Technologien haben unterschiedliche Aufgaben. Dadurch muss nicht alles selbst programmiert werden.

2. Zod – Datenvalidierung

Zod ist für die Überprüfung deiner Formulardaten zuständig.

Es kontrolliert, ob die eingegebenen Daten den vorgegebenen Regeln entsprechen.

2.1 Das Zod-Schema

Dein Code:

typescript

```typescript
const anfo_contact = z.object({
  vorname: z.string().trim().min(1, "Vorname ist erforderlich"),
  nachname: z.string().trim().min(1, "Nachname ist erforderlich"),
  adresse: z.string().trim().min(1, "Adresse ist erforderlich"),
  plz: z
    .string()
    .trim()
    .min(1, "PLZ ist erforderlich")
    .regex(/^[0-9]+$/, "PLZ darf nur Ziffern enthalten"),
  email: z.email("Bitte gib eine gültige E-Mail ein").trim(),
  bdate: z.coerce.date(),
});
```

Was macht dieser Code?

`z.object()` definiert ein Objekt mit bestimmten Eigenschaften.

In deinem Fall ist das Objekt ein Anmeldekontakt.

Eigenschaft	Funktion
`vorname`	Muss ein Text sein und darf nicht leer sein.
`nachname`	Muss ein Text sein und darf nicht leer sein.
`adresse`	Muss ein Text sein und darf nicht leer sein.
`plz`	Darf nur aus Ziffern bestehen.
`email`	Muss eine gültige E-Mail-Adresse sein.
`bdate`	Wird in ein gültiges Datumsobjekt umgewandelt.

Was bedeuten die einzelnen Funktionen?

- `z.string()` – erwartet einen Text.
- `.trim()` – entfernt Leerzeichen am Anfang und Ende.
- `.min(1)` – verlangt mindestens ein Zeichen.
- `.regex()` – überprüft den Text anhand eines bestimmten Musters.
- `z.email()` – überprüft die E-Mail-Adresse.
- `z.coerce.date()` – versucht, einen eingegebenen Wert in ein Datum umzuwandeln.

Beispiel:

Wenn jemand bei der PLZ `12345abc` eingibt, erkennt Zod den Fehler und verhindert die erfolgreiche Formularübermittlung.

2.2 TypeScript-Typ aus Zod erstellen

Dein Code:

typescript

```
export type FormSchema = z.infer<typeof anfo_contact>;
```

Hier passiert etwas sehr Praktisches.

Normalerweise müsste man die Eigenschaften des Formulars zweimal definieren:

1. Einmal für die Validierung.
2. Einmal für TypeScript.

Mit `z.infer` übernimmt TypeScript automatisch die Typen aus deinem Zod-Schema.

Das bedeutet:

Wenn du später eine weitere Eigenschaft hinzufügst, beispielsweise `telefon`, wird diese auch automatisch im TypeScript-Typ berücksichtigt.

Vorteil: Du vermeidest doppelte Definitionen und mögliche Fehler.

3. Formularfelder – Dynamische Erstellung

Dein Code:

typescript

```
const fields: {
  name: keyof FormSchema;
  placeholder: string;
  type?: string;
}[] = [
```

```
  { name: "vorname", placeholder: "Vorname" },
  { name: "nachname", placeholder: "Nachname" },
  { name: "adresse", placeholder: "Adresse" },
  { name: "plz", placeholder: "PLZ" },
  { name: "email", placeholder: "E-Mail" },
  { name: "bdate", placeholder: "", type: "date" },
];
```

Hier definierst du alle Formularfelder zentral.

Anstatt jedes Eingabefeld einzeln zu programmieren, speicherst du die Eigenschaften der Felder in einer Liste.

Was bedeutet `keyof FormSchema`?

Das ist eine TypeScript-Funktion.

Sie erlaubt nur Namen, die tatsächlich im Zod-Schema existieren.

Beispielsweise:

- `vorname` ist erlaubt.
- `nachname` ist erlaubt.
- `email` ist erlaubt.
- `telefonnummer` wäre nicht erlaubt, solange es nicht im Schema existiert.

Was bedeutet `type?: string`?

Das Fragezeichen bedeutet, dass diese Eigenschaft optional ist.

Du musst also nicht bei jedem Feld einen Typ angeben.

Bei deinen normalen Feldern wird automatisch `text` verwendet. Beim Geburtsdatum wird `date` angegeben.

3.1 Formularfelder automatisch rendern

typescript

```
const renderFields = (
  registerField: typeof register | typeof registerEdit,
  fieldErrors: typeof errors | typeof editErrors,
) =>
  fields.map(({ name, placeholder, type }) => (
    <div key={name}>
      <input
```

```
        className="inputfielsec"
        type={type || "text"}
        placeholder={placeholder}
        {...registerField(name)}
      />
      {fieldErrors[name] && (
        <p className="error">{fieldErrors[name]?.message}</p>
      )}
    </div>
  ));
```

Das ist eine der wichtigsten Verbesserungen an deinem Code.

Früher hast du jedes Eingabefeld einzeln geschrieben.

Jetzt übernimmt `fields.map()` diese Arbeit.

Was passiert dabei?

1. `fields.map()` geht jedes Element deiner Feldliste durch.
2. Für jedes Element wird ein Eingabefeld erstellt.
3. `registerField(name)` verbindet das Eingabefeld mit React Hook Form.
4. `fieldErrors[name]` überprüft, ob ein Validierungsfehler vorhanden ist.
5. Wenn ein Fehler vorhanden ist, wird die Fehlermeldung angezeigt.

Warum ist das sinnvoll?

Du verwendest dieselbe Funktion sowohl für das normale Anmeldeformular als auch für das Bearbeitungsformular.

Dadurch musst du die Eingabefelder nicht zweimal programmieren.

4. React Hook Form – Formularverwaltung

React Hook Form übernimmt die Verwaltung deiner Formulare.

Es kümmert sich unter anderem um:

- Erfassen der Eingaben.
- Verknüpfung der Eingabefelder.
- Übermittlung der Formulardaten.
- Verarbeitung der Validierungsfehler.

Du verwendest dafür `useForm()`.

typescript

```
const {
  register,
  handleSubmit,
  formState: { errors },
} = useForm({
  resolver: zodResolver(anfo_contact),
});
```

Die wichtigsten Funktionen:

Funktion	Aufgabe
`register`	Verbindet ein Eingabefeld mit dem Formular.
`handleSubmit`	Verarbeitet das Absenden des Formulars.
`errors`	Enthält die Validierungsfehler.
`reset`	Setzt Formularfelder auf bestimmte Werte zurück.
`resolver`	Verbindet React Hook Form mit Zod.

Warum gibt es zwei `useForm()`?

Du hast zwei Formulare:

1. Das normale Anmeldeformular.
2. Das Bearbeitungsformular im Popup.

Beide benötigen eine eigene Formularverwaltung.

Deshalb gibt es auch zwei Instanzen:

- `register` für das Anmeldeformular.
- `registerEdit` für das Bearbeitungsformular.

Beide verwenden dasselbe Zod-Schema.

Das ist praktisch, weil du die Validierungsregeln nicht doppelt schreiben musst.

5. Strapi – Datenbank und API

Strapi ist dein Backend.

Dort werden die Anmeldedaten gespeichert und verwaltet.

Deine Anwendung kommuniziert über die Strapi-API mit der Datenbank.

Für deine Anwendung benötigst du vier grundlegende Funktionen:

Funktion	HTTP-Methode	Aufgabe
GET	GET	Daten aus Strapi abrufen
POST	POST	Neue Daten speichern
PUT	PUT	Bestehende Daten bearbeiten
DELETE	DELETE	Daten löschen

Diese Funktionen werden über deine Hooks angesprochen.

Wichtig: Strapi übernimmt die tatsächliche Datenbankverarbeitung. Deine Next.js-Seite muss nicht direkt mit der Datenbank kommunizieren.

6. Axios – Kommunikation mit Strapi

Axios ist eine Bibliothek, mit der du HTTP-Anfragen erstellen kannst.

Beispielsweise:
typescript

```typescript
const api = axios.create({
  baseURL: "http://localhost:1337/api",
});
```

Damit legst du eine zentrale Adresse für deine Strapi-API fest.

Anschließend kannst du die verschiedenen Anfragen durchführen:
typescript

```typescript
api.get("/anfo-contacts");
```

Ruft alle Kontakte ab.
typescript

```typescript
api.post("/anfo-contacts", {
  data: contact,
});
```

Erstellt einen neuen Kontakt.
typescript

```
api.put(`/anfo-contacts/${documentId}`, {
  data: contact,
});
```

Bearbeitet einen bestehenden Kontakt.
typescript

```
api.delete(`/anfo-contacts/${documentId}`);
```

Löscht einen Kontakt.

Vorteil: Du musst nicht jedes Mal die vollständige API-Adresse angeben.

7. TanStack React Query – Verwaltung der Daten

React Query ist dafür zuständig, deine Datenabfragen und Datenänderungen zu verwalten.

Du verwendest dafür sogenannte Hooks.

Ein Hook ist eine wiederverwendbare Funktion, die React-Funktionalität bereitstellt.

Deine Anwendung besitzt vier wichtige Hooks:

Hook	Aufgabe
`useGetanfo_contact`	Ruft Kontakte aus Strapi ab.
`useCreateanfo_contact`	Erstellt einen neuen Kontakt.
`useUpdateanfo_contact`	Bearbeitet einen Kontakt.
`useDeleteanfo_contact`	Löscht einen Kontakt.

7.1 GET – Daten abrufen

Wenn deine Seite geöffnet wird, ruft dieser Hook die Kontakte ab.

React Query verwaltet dabei automatisch den Ladezustand und mögliche Fehler.

Beispielsweise:
typescript

```
const {
  data: contacts,
  isLoading,
```

```
  isError,
} = useGetanfo_contact();
```

Die Variablen bedeuten:

- `contacts`: Enthält die abgerufenen Kontakte.
- `isLoading`: Zeigt an, dass die Daten noch geladen werden.
- `isError`: Zeigt an, dass beim Abrufen ein Fehler aufgetreten ist.

7.2 POST – Neue Daten speichern

typescript

```
const { mutate, isSuccess } = useCreateanfo_contact();
```

`mutate()` startet die Übermittlung der Daten.

Beispielsweise:

typescript

```
const onSubmit = (data: FormSchema) => mutate(data);
```

`isSuccess` zeigt an, ob die Übermittlung erfolgreich war.

7.3 UPDATE – Daten bearbeiten

typescript

```
const {
  mutate: updateContact,
  isPending: isUpdating,
} = useUpdateanfo_contact();
```

Hier wird ein bestehender Datensatz aktualisiert.

`isPending` zeigt an, ob die Änderung gerade durchgeführt wird.

Nach erfolgreicher Bearbeitung wird das Popup geschlossen.

typescript

```
{ onSuccess: () => setSelectedContact(null) }
```

7.4 DELETE – Daten löschen

typescript

```
const onDelete = () => {
  if (!deleteContactData) return;

  deleteContact(deleteContactData.documentId, {
    onSuccess: () => setDeleteContactData(null),
  });
};
```

Hier passiert Folgendes:

1. Es wird überprüft, ob ein Datensatz zum Löschen ausgewählt wurde.
2. Die `documentId` wird an den DELETE-Hook übergeben.
3. Strapi löscht den Datensatz.
4. Bei erfolgreicher Löschung wird das Bestätigungsfenster geschlossen.

8. Die beiden Popups

Du hast zwei verschiedene Popups eingebaut.

8.1 Bearbeitungs-Popup

Dafür verwendest du:

typescript

```
const [selectedContact, setSelectedContact] =
  useState<AnfoContact | null>(null);
```

Das ist ein React-State.

Er speichert den Kontakt, den du bearbeiten möchtest.

Wenn du auf das Stiftsymbol klickst, wird der ausgewählte Kontakt gespeichert.

typescript

```
const handleEdit = (contact: AnfoContact) => {
  setSelectedContact(contact);
  resetEdit({
    vorname: contact.vorname,
    nachname: contact.nachname,
    adresse: contact.adresse,
    plz: contact.plz,
    email: contact.email,
    bdate: contact.bdate.slice(0, 10),
```

```
  });
};
```

`resetEdit()` befüllt das Bearbeitungsformular automatisch mit den vorhandenen Daten.

Du kannst die Daten verändern und anschließend speichern.

8.2 Lösch-Bestätigungs-Popup

Dafür verwendest du:

typescript

```
const [deleteContactData, setDeleteContactData] =
  useState<AnfoContact | null>(null);
```

Dieser State speichert den Kontakt, der gelöscht werden soll.

Im Popup kannst du dadurch beispielsweise Folgendes anzeigen:

tsx

```
Möchtest du folgenden Eintrag wirklich löschen?

{deleteContactData?.vorname}
{deleteContactData?.nachname}
```

Erst wenn der Benutzer auf Löschen klickt, wird `onDelete()` aufgerufen.

Wenn er auf Abbrechen klickt, wird der State wieder auf `null` gesetzt.

Warum ist das besser als `window.confirm()`?

- Du kannst das Fenster selbst gestalten.
- Du kannst Vor- und Nachnamen anzeigen.
- Du kannst eigene Schaltflächen verwenden.
- Du kannst das Aussehen an dein restliches Design anpassen.

9. TypeScript – Warum wir es verwenden

TypeScript erweitert JavaScript um ein Typsystem.

Es hilft dir dabei, Fehler bereits während der Entwicklung zu erkennen.

Beispielsweise:

typescript

```
const onDelete = (documentId: string) => {
  // ...
};
```

Hier definierst du, dass `documentId` ein Text sein muss.

Wenn du versehentlich eine Zahl oder ein anderes Objekt übergibst, kann TypeScript einen Fehler anzeigen.

Ein weiteres Beispiel:
typescript

```
const [selectedContact, setSelectedContact] =
  useState<AnfoContact | null>(null);
```

Das bedeutet:

- `selectedContact` kann einen Kontakt enthalten.
- Alternativ kann der Wert `null` sein.

TypeScript weiß dadurch genau, welche Eigenschaften dein Kontakt besitzt.

Das ist besonders bei der Kommunikation zwischen Strapi und deiner Anwendung hilfreich.

10. Was wurde durch die Optimierung verbessert?

Ursprünglich hattest du viele einzelne Eingabefelder und wiederholte Codeabschnitte.

Durch die Änderungen haben wir deinen Code übersichtlicher gestaltet.

Vorher	Jetzt
Jedes Eingabefeld einzeln geschrieben	Zentrale Felddefinition
Wiederholte Formularfelder	Eine Renderfunktion
Validierung separat definiert	Ein gemeinsames Zod-Schema
Zwei Formulare mit eigenen Felddefinitionen	Gemeinsame Felddefinition für beide
Browser-Bestätigung beim Löschen	Eigenes Lösch-Popup
Einzelne API-Aufrufe in der Page	Wiederverwendbare Hooks

Manuelle Verwaltung der Datenabfragen	React Query

11. Welche Technologie übernimmt welche Aufgabe?

Das ist eine besonders wichtige Unterscheidung, die du dir merken solltest:

Technologie	Zuständigkeit
TypeScript	Typen und Fehlererkennung
Zod	Datenvalidierung
React Hook Form	Verwaltung der Formulare
React	Benutzeroberfläche und States
Axios	Kommunikation mit der API
TanStack React Query	Verwaltung der Datenabfragen und Änderungen
Strapi	Backend und Datenbankverwaltung
Tailwind CSS	Gestaltung der Benutzeroberfläche
Lucide React	Icons für Bearbeiten und Löschen

12. Was solltest du bei zukünftigen Erweiterungen beachten?

Dein Aufbau ist bereits eine gute Grundlage für weitere Funktionen.

Ich würde dir empfehlen, diese Struktur beizubehalten:

text

```
app/
│
├── page.tsx
│
└── hooks/
    ├── usepost_anfo_contact.ts
    ├── useget_anfo_contact.ts
    ├── useUpdate_anfo_contact.ts
    └── useDelete_anfo_contact.ts
```

Die Aufgaben sollten weiterhin getrennt bleiben:

- `page.tsx`: Darstellung, Formulare und Benutzerinteraktionen.

- `hooks/`: Kommunikation mit Strapi und React Query.
- Zod-Schema: Validierung der Eingaben.

> **Eine wichtige Ergänzung für später:** Deine Zod-Validierung im Frontend schützt nicht automatisch deine Strapi-Datenbank. Strapi sollte dieselben wichtigen Regeln zusätzlich serverseitig überprüfen. Außerdem solltest du bei einer Veröffentlichung deiner Anwendung die API-Adresse nicht auf `localhost` belassen und Schreibzugriffe entsprechend absichern.